COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/02/2026	**DATE SUBMITTED:**	10/02/2026
NAME:	KEITH NICOLE SILVOZA, CYRILL JOFERSON REYES	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 10.1 GRAPHS

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp | ma

C: > Users > TIPQC > Downloads > graph_SILVOZA.cpp > ...

```cpp
#include <iostream>
// stores adjacency list items
struct adjNode {
int val, cost;
adjNode* next;
};
// structure to store edges
struct graphEdge {
int start_ver, end_ver, weight;
};
class DiaGraph{
// insert new nodes into adjacency list from given graph
adjNode* getAdjListNode(int value, int weight, adjNode* head) {
adjNode* newNode = new adjNode;
newNode->val = value;
newNode->cost = weight;
newNode->next = head; // point new node to current head
return newNode;
}
int N; // number of nodes in the graph
public:
adjNode **head; //adjacency list as array of pointers
// Constructor
DiaGraph(graphEdge edges[], int n, int N) {
// allocate new node
head = new adjNode*[N]();
this->N = N;
// initialize head pointer for all vertices
for (int i = 0; i < N; ++i)
head[i] = nullptr;
// construct directed graph by adding edges to it
for (unsigned i = 0; i < n; i++) {
int start_ver = edges[i].start_ver;
int end_ver = edges[i].end_ver;
int weight = edges[i].weight;
// insert in the beginning
adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
// point head pointer to new node
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp

C: > Users > TIPQC > Downloads > graph_SILVOZA.cpp > ...

```
11    class DiaGraph{
21    public:
24    DiaGraph(graphEdge edges[], int n, int N) {
32    for (unsigned i = 0; i < n; i++) {
36    // insert in the beginning
37    adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
38    // point head pointer to new node
39    head[start_ver] = newNode;
40    }
41    }
42    // Destructor
43    ~DiaGraph() {
44    for (int i = 0; i < N; i++) {
45    adjNode* curr = head[i];
46    while (curr != nullptr) {
47    adjNode* nextNode = curr->next;
48    delete curr;
49    curr = nextNode;
50    }
51    }
52    delete[] head;
53    }
54    };
55    // print all adjacent vertices of given vertex
56    void display_AdjList(adjNode* ptr, int i)
57    {
58    while (ptr != nullptr) {
59    std::cout << "(" << i << ", " << ptr->val
60    << ", " << ptr->cost << ") ";
61    ptr = ptr->next;
62    }
63    std::cout << std::endl;
64    }
65    // graph implementation
66    int main()
67    {
68    // graph edges array.
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp | main.cpp

C: > Users > TIPQC > Downloads > graph_SILVOZA.cpp > ...

```
11    class DiaGraph{
21    public:
43    ~DiaGraph() {
54    };
55    // print all adjacent vertices of given vertex
56    void display_AdjList(adjNode* ptr, int i)
57    {
58    while (ptr != nullptr) {
59    std::cout << "(" << i << ", " << ptr->val
60    << ", " << ptr->cost << ") ";
61    ptr = ptr->next;
62    }
63    std::cout << std::endl;
64    }
65    // graph implementation
66    int main()
67    {
68    // graph edges array.
69    graphEdge edges[] = {
70    // (x, y, w) -> edge from x to y with weight w
71    {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
72    };
73    int N = 6; // Number of vertices in the graph
74    // calculate number of edges
75    int n = sizeof(edges)/sizeof(edges[0]);
76    // construct graph
77    DiaGraph diagraph(edges, n, N);
78    // print adjacency list representation of graph
79    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;
80    for (int i = 0; i < N; ++i) {
81    display_AdjList(diagraph.head[i], i);
82    }
83
84    return 0;
85    }
```

Analysis Procedure

In ILO A, we developed a directed, weighted graph known as DiaGraph in C++ by utilizing an adjacency list. Rather than opting for a 2D matrix, which consumes additional memory, we implemented an array of head pointers (adjNode head) where each index corresponds to an originating vertex. While adding edges, our program dynamically generated an adjNode that represented the destination vertex and weight, subsequently placing it at the start of that vertex's linked list through getAdjListNode(). The function display_AdjList() was capable of printing the list, effectively displaying all connected adjacent nodes in the format (start_vertex, end_vertex, weight). Furthermore, we created a specialized destructor (~DiaGraph()) that traverses each linked list to remove every node before disposing of the main head array, thereby safeguarding against memory leaks.

ILO B

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp

C: > Users > TIPQC > Downloads > graph.hpp > ...

```cpp
#ifndef GRAPH_HPP
#define GRAPH_HPP

#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>

// Forward declaration
template <typename T>
class Graph;

// Structure representing a graph edge
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

// Graph class definition
template <typename T>
class Graph {
public:
    explicit Graph(size_t N) : V(N) {}

    size_t vertices() const { return V; }
    const std::vector<Edge<T>>& edges() const { return edge_list; }
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp

C: > Users > TIPQC > Downloads > graph.hpp > ...

```
1   #ifndef GRAPH_HPP
31  class Graph {
32  public:
38      void add_edge(Edge<T> &&e) {
39          if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
41          } else {
42              std::cerr << "Vertex out of bounds" << std::endl;
43          }
44      }
45
46      std::vector<Edge<T>> outgoing_edges(size_t v) const {
47          std::vector<Edge<T>> edges_from_v;
48          for (auto &e : edge_list) {
49              if (e.src == v) {
50                  edges_from_v.emplace_back(e);
51              }
52          }
53          return edges_from_v;
54      }
55
56      template <typename U>
57      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
58
59  private:
60      size_t V;
61      std::vector<Edge<T>> edge_list;
62  };
63
64  // Stream insertion operator for Graph
65  template <typename T>
66  std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
67      for (size_t i = 1; i < G.vertices(); i++) {
68          os << i << ": \t";
69          auto edges = G.outgoing_edges(i);
70          for (auto &e : edges) {
71              os << " " << e.dest << ": " << e.weight << " , ";
72          }
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp | main.

C: > Users > TIPQC > Downloads > graph.hpp > ...

```
1   #ifndef GRAPH_HPP
66  std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
67      for (size_t i = 1; i < G.vertices(); i++) {
70          for (auto &e : edges) {
73              os << std::endl;
74      }
75      return os;
76  }
77
78  // Depth-First Search algorithm implementation
79  template <typename T>
80  std::vector<size_t> depth_first_search(const Graph<T> &G, size_t start_vertex = 1) {
81      std::stack<size_t> stack;
82      std::vector<size_t> visit_order;
83      std::set<size_t> visited;
84
85      stack.push(start_vertex);
86
87      while (!stack.empty()) {
88          auto current_vertex = stack.top();
89          stack.pop();
90
91          if (visited.find(current_vertex) == visited.end()) {
92              visited.insert(current_vertex);
93              visit_order.push_back(current_vertex);
94
95              for (auto e : G.outgoing_edges(current_vertex)) {
96                  if (visited.find(e.dest) == visited.end()) {
97                      stack.push(e.dest);
98                  }
99              }
100         }
101     }
102     return visit_order;
103 }
104
105 // Helper function to generate reference graph
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp | main.cp

C: > Users > TIPQC > Downloads > graph.hpp > ...

```
1   #ifndef GRAPH_HPP
80  std::vector<size_t> depth_first_search(const Graph<T> &G, size_t start_vertex = 1) {
87      while (!stack.empty()) {
91          if (visited.find(current_vertex) == visited.end()) {
95              for (auto e : G.outgoing_edges(current_vertex)) {
100             }
101         }
102     return visit_order;
103 }
104
105 // Helper function to generate reference graph
106 template <typename T>
107 Graph<T> create_reference_graph() {
108     Graph<T> G(9);
109     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
110     edges[1] = {{2, 0}, {5, 0}};
111     edges[2] = {{1, 0}, {5, 0}, {4, 0}};
112     edges[3] = {{4, 0}, {7, 0}};
113     edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
114     edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
115     edges[6] = {{4, 0}, {7, 0}, {8, 0}};
116     edges[7] = {{3, 0}, {6, 0}};
117     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
118
119     for (auto &i : edges) {
120         for (auto &j : i.second) {
121             G.add_edge(Edge<T>{i.first, j.first, j.second});
122         }
123     }
124     return G;
125 }
126
127 #endif // GRAPH_HPP
```

graph_SILVOZA.cpp graph.hpp B.2.h

C: > Users > TIPQC > Downloads > main.cpp

```cpp
#include "Graph.hpp"
#include <iostream>

template <typename T>
void test_DFS() {
    auto G = create_reference_graph<T>();
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order) {
        std::cout << v << std::endl;
    }
}

int main() {
    using T = unsigned;
    test_DFS<T>();

```

Analysis Procedure

We employed an iterative algorithm for Depth-First Search, initiating our process at Vertex 1. An explicit stack (std::stack) was utilized to manage the traversal, and we maintained a std::set to monitor which nodes had been visited to prevent the program from getting trapped in endless loops due to graph cycles. Since a stack functions on a Last-In, First-Out (LIFO) principle, adding unvisited neighboring vertices to the stack compelled the algorithm to delve as deeply as possible along one path before reverting. Executing the program yielded the visiting order of 1 to 5 to 8 to 6 to 7 to 3 to 4 to 2, illustrating the anticipated deep-first search behavior.

ILO B.2

HOA10 > iloB.2 > main.cpp > ...

```cpp
#include "Graph.h"
#include <iostream>

// Step 7: Driver and test function
template <typename T>
void test_BFS()
{
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

HOA10 > iloB.2 > C Graph.h > operator<<<T>(std::ostream &, const Graph<T>&)

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

// Step 1: Forward declaration
template <typename T>
class Graph;

// Step 2: Edge structure
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Forward declaration of stream insertion operator
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

// Step 4: Graph class definition
template <typename T>
class Graph
{
public:
    explicit Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

```

```
41  public:
48
49      auto &edges() const
50      {
51          return edge_list;
52      }
53
54      void add_edge(Edge<T> &&e)
55      {
56          if (e.src >= 1 && e.src <= V &&
57              e.dest >= 1 && e.dest <= V)
58              edge_list.emplace_back(e);
59          else
60              std::cerr << "Vertex out of bounds" << std::endl;
61      }
62
63      auto outgoing_edges(size_t v) const
64      {
65          std::vector<Edge<T>> edges_from_v;
66          for (auto &e : edge_list)
67          {
68              if (e.src == v)
69              {
70                  edges_from_v.emplace_back(e);
71              }
72          }
73          return edges_from_v;
74      }
75
76      template <typename U>
77      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
78
79  private:
80      size_t V;
81      std::vector<Edge<T>> edge_list;
82  };
83
```

```cpp
// Step 3: Overloaded << operator definition
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (size_t i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

// Step 5: Function to create reference graph
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

```

```
119
120 // Step 6: Breadth-First Search algorithm
121 template <typename T>
122 auto breadth_first_search(const Graph<T> &G, size_t start_vertex = 1)
123 {
124     std::queue<size_t> queue;
125     std::vector<size_t> visit_order;
126     std::set<size_t> visited;
127
128     queue.push(start_vertex);
129
130     while (!queue.empty())
131     {
132         auto current_vertex = queue.front();
133         queue.pop();
134
135         if (visited.find(current_vertex) == visited.end())
136         {
137             visited.insert(current_vertex);
138             visit_order.push_back(current_vertex);
139
140             for (auto e : G.outgoing_edges(current_vertex))
141                 queue.push(e.dest);
142         }
143     }
144     return visit_order;
145 }
146
147 #endif // GRAPH_H
```

```
PS C:\Users\TIPQC\OneDrive\Documents\CPE12S4 - R
ngine-In-x0c1gxqe.2kc' '--stdout=Microsoft-MIEng

1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

C: > Users > TIPQC > Downloads > C B.2.h > ...

```h
#ifndef GRAPH_H
#define GRAPH_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

// Step 1: Forward declaration
template <typename T>
class Graph;

// Step 2: Edge structure
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Forward declaration of stream insertion operator
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

// Step 4: Graph class definition
template <typename T>
class Graph
{
public:
    explicit Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp

C: > Users > TIPQC > Downloads > B.2.h > ...

```
1   #ifndef GRAPH_H
39  class Graph
41  public:
44      auto vertices() const
47      }
48
49      auto &edges() const
50      {
51          return edge_list;
52      }
53
54      void add_edge(Edge<T> &&e)
55      {
56          if (e.src >= 1 && e.src <= V &&
57              e.dest >= 1 && e.dest <= V)
58              edge_list.emplace_back(e);
59          else
60              std::cerr << "Vertex out of bounds" << std::endl;
61      }
62
63      auto outgoing_edges(size_t v) const
64      {
65          std::vector<Edge<T>> edges_from_v;
66          for (auto &e : edge_list)
67          {
68              if (e.src == v)
69              {
70                  edges_from_v.emplace_back(e);
71              }
72          }
73          return edges_from_v;
74      }
75
76      template <typename U>
77      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
78
79  private:
80      size_t V;
81      std::vector<Edge<T>> edge_list;
82  };
83
84  // Step 3: Overloaded << operator definition
85  template <typename T>
86  std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
87  {
88      for (size_t i = 1; i < G.vertices(); i++)
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp

C: > Users > TIPQC > Downloads > B.2.h > ...

```
1   #ifndef GRAPH_H
86  std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
87  {
88      for (size_t i = 1; i < G.vertices(); i++)
89      {
90          os << i << ":\t";
91          auto edges = G.outgoing_edges(i);
92          for (auto &e : edges)
93              os << "{" << e.dest << ": " << e.weight << "}, ";
94          os << std::endl;
95      }
96      return os;
97  }
98
99  // Step 5: Function to create reference graph
100 template <typename T>
101 auto create_reference_graph()
102 {
103     Graph<T> G(9);
104     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
105     edges[1] = {{2, 2}, {5, 3}};
106     edges[2] = {{1, 2}, {5, 5}, {4, 1}};
107     edges[3] = {{4, 2}, {7, 3}};
108     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
109     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
110     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
111     edges[7] = {{3, 3}, {6, 4}};
112     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
113
114     for (auto &i : edges)
115         for (auto &j : i.second)
116             G.add_edge(Edge<T>{i.first, j.first, j.second});
117     return G;
118 }
119
120 // Step 6: Breadth-First Search algorithm
121 template <typename T>
122 auto breadth_first_search(const Graph<T> &G, size_t start_vertex = 1)
123 {
124     std::queue<size_t> queue;
125     std::vector<size_t> visit_order;
126     std::set<size_t> visited;
127
128     queue.push(start_vertex);
129
130     while (!queue.empty())
```

graph_SILVOZA.cpp | graph.hpp | B.2.h | b.2_main.cpp | mai

C: > Users > TIPQC > Downloads > B.2.h > ...

```
#ifndef GRAPH_H
auto create_reference_graph()
    edges[1] = {{2, 2}, {5, 5}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

// Step 6: Breadth-First Search algorithm
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t start_vertex = 1)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(start_vertex);

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

#endif
```

Analysis Procedure

In section B.2, we applied Breadth-First Search starting at Vertex 1. The primary change was substituting the stack with a queue (std::queue). As a queue operates on a First-In, First-Out (FIFO) principle, it compelled the exploration to proceed through the graph in levels (or layers). Whenever a node was removed from the front of the queue, all of its directly unvisited neighbors were added to the rear. This guaranteed that all neighbors at distance 1 were explored prior to addressing those at distance 2. The output of our execution presented the visiting sequence 1 → 2 → 5 → 4 → 8 → 3 → 6 → 7, affirming a layer-by-layer exploration.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
   Depth-First Search (DFS)
   - The DFS algorithm is ideal for this situation. It fundamentally works by going as deep as possible along a path before reversing its direction. Utilizing either recursion or a stack structure, DFS delves deep into a pathway until it encounters a point where there are no further options, then it retraces its steps to the last vertex that has remaining alternatives and resumes the exploration from there.
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

   Psuedocode:

   ALGORITHM ExamineTreeDFS(root):
   1. IF root is NULL:
   RETURN

   2. PROCESS(root.value) // Step A: Visit current node first

   3. ExamineTreeDFS(root.left) // Step B: Recursively go as deep as possible down left child
   4. ExamineTreeDFS(root.right) // Step C: Backtrack and explore right child
   END ALGORITHM

   Step A (Processing First): The moment the function arrives at a node, it immediately prints or processes its value right away before moving anywhere else.

   Step B (Diving Deep): Instead of checking neighbor nodes at the same level, the function pauses where it is and pushes a new call onto the stack. It keeps diving as deep as possible down the left branch until it hits a NULL leaf node.

   Step C (Backtracking): Once it hits the end of a branch, the current function frame pops off the call stack. This automatically backtracks it to the parent node, allowing it to turn around and explore the right branch.
   ALGORITHM ExamineTreeDFS(root):
   1. IF root is NULL:
   RETURN

2. PROCESS(root.value) // Step A: Visit current node first

3. ExamineTreeDFS(root.left) // Step B: Recursively go as deep as possible down left child
4. ExamineTreeDFS(root.right) // Step C: Backtrack and explore right child
END ALGORITHM

3. In the performed code, what data structure is used to implement the Breadth First Search?

- In the execution, a Queue (std::queue) is utilized to carry out Breadth-First Search. The First-In, First-Out (FIFO) characteristic of a queue ensures that nodes are investigated in layers, precisely in the sequence they were found.

4. How many times can a node be visited in the BFS?

- In a correctly executed BFS using a visited collection or list, every node is processed/visited a single time. Verifying the visit status prior to adding or removing a vertex guarantees that the method does not revisit nodes or get caught in endless loops within graph cycles.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **GEMINI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, fixing C++ syntax errors, formatting output tables, and organizing answers for the lab report.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to assist in debugging syntax errors in the provided starter code, formatting C++ code snippets, and structuring output tables cleanly for the lab report. All graph data structure implementations, manual traversal tracing, code testing, and final analysis were performed and verified independently by the team. The AI served solely as a formatting and assistance tool and did not replace original thinking, analysis, or completion of the required work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.